eGrant tagove funkcie

---

active_language(): *string*

returns the active user language

---

active_user(*string* tag, *bool*|*string* format = *false*): *mixed*

returns the `tag` value from the active user

---

all_custom_form_tags(*string* form_tag, *string* tag, *boolean* includeDrafts = *false*, *boolean* includeRejected = *false*): *array*

returns an array of `tag` value from all custom forms with the tag `form_tag` that are not in DRAFT state for the active application. If `includeDrafts` is `true`, also drafts are included. If `includeRejected` is `true`, also rejected forms are included

---

all_inter_tags(*string* tag, *boolean* includeDrafts = *false*, *boolean* includeRejected = *false*): *array*

returns an array of `tag` value from all intermediate reports for the active application that are not in DRAFT state. If `includeDrafts` is `true`, also drafts are included. If `includeRejected` is `true`, also rejected reports are included

---

and(*boolean* val1, *boolean* val2, ...): *boolean*

returns the logical AND from all input arguments, a `null` argument ends the list

---

app(*int* id, *string* tag, *bool*|*string* format = *false*): *mixed*

returns the `tag` value from application with `id`

---

application_counter(*string* tag = *"default"*): *?int*

return next value from the counter with `tag` of the current application, every application keeps its own series of values for each `tag`

---

array(*mixed* value1, *mixed* value2, ...): *array*

create an array from provided items. If an item is an array with a single key-value pair, the key-value pair is added to the resulting array. The array function is often used together with the item function to create key-value pairs.

---

array_cartesian(*array* input, *string* key, *bool* addEmptyValue = 1): *array*

expand each item of `input` into one item per value of its `key` array (the `key` holding that single value), adding an item with an empty `key` value if `addEmptyValue` is true

---

array_column(*array* input, *mixed* columnKey, *mixed* indexKey = null): *array*

return the values from the `columnKey` column in the `input` array, optionally indexed by the `indexKey` column

---

array_diff(*array* array1, *array* array2, ...): *array*

returns the values of `array1` not present in any of the other arrays, a `null` argument ends the list

---

array_filter(*array* input, *string* program): *array*

filter the array `input` by running `program` on each item and returning only those items for which `program` returns true. For each iteration of `program`, the values from the array item are expanded to local variables and the whole item is available under the `__data` key.

---

array_find(*array* input, *string* search_key = *""*, *mixed* search_value = *""*, *string* result_key = *null*): *mixed*

returns the first item of array `input` which has `search_key` value equal to `search_value`, or null. If `result_key` is defined, it returns only that key from the found item, otherwise the whole item is returned. If `search_key` is empty, the items themselves are compared with `search_value`

---

array_flatten(*array* input, *bool* preserveKeys = 0): *array*

flatten an array of arrays to a single array, if `preserveKeys` is true, the keys are preserved

---

array_key_by(*array* value, *string* key): *array*

create keys in array `value` by the values from `key`

---

array_keys(*array* value): *array*

returns only the keys from `value`

---

array_make(*mixed* value, *mixed* key = null): *array*

create a simple array from value and optional key

---

array_map(*array* input, *string* program, *array* input2 = *null*): *array*

run `program` on each item from `input` and replace it with the result. For each iteration of `program`, the values from the array item are expanded to local variables. Inside the `program`, there are two special variables available:

- `__data` - the current item from the array
- `__key` - the key of the current item in the array

When `input2` is provided, both arrays are walked in parallel: `__data2` holds the item of `input2`, `__key` is not available and the result is reindexed.

---

array_mean(*array* array, *string*|*null* key = null): *float*|*null*

returns the mean (average) of numeric values in `array`. if `key` is provided, extracts that key from each sub-array element before averaging. non-numeric values are ignored. returns null if there are no numeric values.

---

array_merge(*array* array1, *array* array2, ...): *array*

merges two or more arrays, a `null` argument ends the list

---

array_merge_by_key(*array* array1, *array* array2, *string* key, *bool* strict = false): *array*

merges `array1` with values from `array2` using `key` as the merge key, values not paired from `array2` are not included in the result

---

array_merge_recursive(*array* array1, *array* array2, *bool* preserveNumericKeys = false): *array*

merges two arrays recursively

---

array_non_empty_rows(*array* input): *array*

returns only those rows from the `input` array which are not empty — an array row is empty when all of its values are empty (null, false, 0 or ""), any other row is empty when the value itself is empty. On non-array `input` an empty array is returned.

---

array_only_numeric_keys(*array* input): *array*

returns only those items from the `input` array which have numeric keys

---

array_reduce(*array* input, *string* program, *mixed* initial_value = *null*): *mixed*

reduce the array `input` by running `program` for each item. For each iteration of `program`, the values from the array item are expanded to local variables. Also the result of each iteration is stored into the special variable `__carry`, a non-array item is available as `__data`

---

array_set(*array* source, *string* key, *mixed* value): *array*

set the `key` in array `source` to `value`, `key` can be a dot notation for nested items

---

array_slice(*array* array, *int* offset, *int* length = null): *array*

returns a slice of the provided array starting at `offset` with optional `length`

---

array_sort(*array* value, *string* key = *""*, *enum(asc,desc)* direction = *"asc"*): *array*

sorts the array `value` by provided `key` or by value if `key` is empty, keys are preserved

---

array_sum(*array* input, *string* key = *null*): *numeric*

sum the values from array `input` using the `key` value from each item or sum the items directly if `key` is empty

---

array_values(*array* value): *array*

returns only the values from `value` discarding any array keys

---

assign(*string* variable, *mixed* value): *mixed*

assign the value `value` to `variable` and return it

---

average_array(*string* key, *array* array1, *array* array2, ...): *array*

return `array1` with the `key` value of each item averaged with the items under the same index in the other arrays

---

base64_decode(*string* input): *string*

decode the base64 encoded string `input` to string, returns `false` for invalid input

---

cache(*string* key, *mixed* value, *int* ttlSeconds = 60): *mixed*

caches the `value` under `key` for `ttlSeconds` seconds and returns it. If the `key` is already cached, the cached value is returned instead. If `value` is a function call, it will be only evaluated if the `key` is not cached. A `null` value is not cached. Keys are scoped to the current instance.

---

cache_forget(*string* key): *void*

removes the cached value stored under `key`

---

call(*string* method, ...): *mixed*

calls the custom defined function `method` with provided parameters and returns the result

---

ceil(*numeric* value): *numeric*

return the upper integer bound of the `value`

---

change_form_preview(): *bool*

returns true when the value is being evaluated as part of a change-form preview PDF - the application PDF generated "as if" an unfinished/unaccepted change form were already applied

---

change_form_tag(*string* tag, *bool*|*string* format = *false*): *mixed*

returns the `tag` value from the change form instance driving the current change-form preview PDF render, returns an empty string when the current render is not a change-form preview

---

changed(*string* tag): *bool*

returns true when the item's data differs between the two sides of the diff being rendered, compared exactly the way the diff view itself compares it

available while a DIFF is generated - a change-form or application-version diff view/PDF, or a DIFF output template

---

changed_children(*string* tag): *array*

returns the full tag paths of the direct children of `tag` whose data changed, in form order; each returned path can be passed back to `changed_children`, `changed`, `old` or `new`

available while a DIFF is generated - a change-form or application-version diff view/PDF, or a DIFF output template

---

coalesce(*mixed* val1, *mixed* val2, ...): *mixed*

returns the first non NULL or FALSE parameter

---

company_search(*string* companyId, *string* country = *instance country*, *string* key = *null*): *mixed*

searches `companyId` in the company register of `country` (`sk`, `cz`, `si`, `ee`, `hr`; the country of the instance when none is given) and returns the found company as the array a company search form item stores after a manual search - every item tag (`name`, `cin`, `formatted_address`, ...) plus `statutory` and `is_valid`, so the result can be written straight into such an item, during an import for instance. `companyId` is normalized the same way the search box normalizes what the user types: all whitespace is dropped. It is returned back under `ico`. When `key` is given, only that value is returned instead of the whole array. Returns null when the company is not in the register, and the register is asked only once per identifier and country in one run.

---

concat(*mixed* val1, *mixed* val2, ...): *string*

returns the string concatenation of all provided parameters

---

count(*array* value): *int*

returns the count of items in the `value` array

---

currency(*numeric* value, *string* currency = *"EUR"*): *string*

format the provided `value` in `currency`

---

custom_data(*string* data_tag, *string* tag, *bool*|*string* format = false): *mixed*

returns the `tag` value for the custom data stored under `data_tag` and defined at /custom-data, if `format` is true, the tag value is formatted

Only custom data built on a form has values to format - an option is returned by the label it is offered under rather than by the key it is stored as, a date the way it is written. Every other kind of custom data holds text as it was entered and returns it unchanged.

A formatted value is meant to be shown, not compared: it is text even where the stored value is a number, and an option whose key the form no longer offers reads as empty rather than as the key. Read the same tag without `format` to decide anything on it.

`format` may also name how a value holding several things is written out - `"list"` for the values themselves, `"map"` for them against what they are stored as, see `tag()`. This carries to the `data` tag, so a whole data set can be read as `{"a": "Áno"}` rather than `"Áno"`.

---

custom_data_item(*string* data_tag, *string* primary_key, *string* tag = '', *bool*|*string* format = false): *mixed*

returns the `tag` value of the single row of the custom data stored under `data_tag` whose primary key is `primary_key` — one row, where `custom_data()` reads the whole data set

Only custom data built on a form has rows and a primary key (the tag the key is generated from is set at /custom-data). A data set of any other kind, one that defines no primary key, and a key no row carries, all return null.

`tag` is read on the row: `__all` for everything it holds, the path of a form item for one value, `id`, `primary_key`, `is_valid`, or `form_instance.<tag>` for the form instance behind it. An empty `tag` describes the row itself.

`format` works as it does in `custom_data()` — the value as it reads rather than as it is stored, and `"list"` / `"map"` for a value holding several things.

---

custom_event_button(*string* buttonTag, *string* buttonLabel, *array* config = [] , moreParams...): *string*

renders a custom button with `buttonTag` and `buttonLabel`, which can be handled by a custom event handler. Additional configuration can be provided in `config` array:

- confirm: *string* - confirmation message to show before executing the button action
- context: *enum(application,program,form_instance)* - context in which the button is executed
- form_instance_id: *int* - with `context: form_instance`, the instance to run against; defaults to the one being filled (`@this`)

Additional parameters can be provided after `config`, these parameter are passed to the button action handler.

---

custom_template_context(*string* context, *mixed* value): *mixed*

executes the `value` in the custom template context of `context`, this is useful when you need to simulate the behavior of a custom template context

---

data(*string* tag): *array*

returns the custom data stored under `tag` and defined at /custom-data

---

date(*string* format = '', *timestamp*|*string* time = *now()*): *string*

formats a time value according to a date format string as specified in https://www.php.net/manual/en/datetime.format.php. The `time` can be a numeric timestamp or a date string, in

which case it is converted to a timestamp via strtotime. An empty `format` uses the default date format; `date(timestamp)` formats the timestamp with the default date format.

---

div(*numeric* val1, *numeric* val2, ...): *numeric*

returns the first value divided by all the following values, zero and non-numeric values are ignored

---

empty(*mixed* value): *bool*

returns `true` if `value` is empty (null, false, 0, "0", "" or an empty array)

---

empty_array(): *array*

returns an empty array

---

eq(*mixed* val1, *mixed* val2): *boolean*

returns true if the values val1 and val2 are equal

---

excel_date_to_timestamp(*numeric* excelDate): *int*|*null*

converts the Excel date number `excelDate` to a Unix timestamp

---

explode(*string* value, *string* delimiter = *", "*): *array*

explodes the string `value` into array using the `delimiter`

---

file_link(*int* file_id, *bool* make_public = false): *string*

returns the download link for the file with `file_id`, if `make_public` is true, the link is public and can be accessed without authentication

---

float(*mixed* value): *numeric*

converts `value` to float, works with `,` and `.` as decimal separators

---

form(*int* id, *string* tag, *bool*|*string* format = *false*): *mixed*

returns the `tag` value from form instance with `id`

---

form_item_children(*string* itemTag, *int* formInstanceId = null): *array*

returns the list of full item tags of the direct children of the form item with `itemTag` from the form instance with `formInstanceId`. If `formInstanceId` is not provided, the form of the current application is used. The children are returned in the order they appear in the form. The returned tags can be passed back as `itemTag` to this function or to form_item_data / render_form_item, as long as the same `formInstanceId` is passed along with them.

---

form_item_data(*string* itemTag, *string* dataTag = null, *int* formInstanceId = null, *int* formId = null): *mixed*

returns the definition data for the form item with `itemTag` from the form instance with `formInstanceId` or `formId`. If `dataTag` is provided, only that key of the item definition data is returned.

---

form_item_key(*mixed* text, *string* itemTag, *int* formId = null, *string* separator = ","): *mixed*

translates the human-readable `text` - typically a label read out of an imported spreadsheet cell - into the internal key the form item with `itemTag` stores for that option. The item is looked up in the form with `formId`, or in the form of the current application when `formId` is not provided; `itemTag` may be passed with or without its `@final.` / `@inter.` / `@form[...]` prefix and with or without row indexes.

Only items offering a fixed list of options are supported - Select, Select in a table, MultiValue (one option) and MultiValue (checkboxes) - no matter whether their options are typed in, taken from a roller, read from another tag or produced by custom code.

A key is looked for in this order: the key itself (so an already converted value passes through), the exact label, then the label and the key compared without case, accents and surrounding whitespace, and finally the label without the roller code prefix.

For an item holding a single value the matching key is returned, or `null` when the text names no option. For an item holding multiple values (MultiValue with checkboxes) the `text` is first split by `separator` and an *array* of the matching keys is returned - texts that match no option are dropped, so the result may be empty. An array passed as `text` is taken as an already split list of labels.

---

format(*string* template, ...): *string*

formats a provided template according to https://www.php.net/manual/en/function.sprintf.php

---

get_application_custom_data(*string* key, *mixed* default = null, *int* application_id = null): *mixed*

returns the data stored under `key` for the application (written by its `set_custom_data` external method). `key` is a dot notation key for nested data. If application_id is not specified, the current application is taken. If no data is stored under `key`, `default` is returned

---

get_instance_custom_data(*string* key, *mixed* default = null): *mixed*

returns the instance data stored under `key`. `key` is a dot notation key for nested data. If no data is stored under `key`, `default` is returned

---

global_counter(*string* tag = *"default"*): *int*

return next value from global instance counter with `tag`

---

glue_concat(*string* glue, *mixed* val1, *mixed* val2, ...): *string*

returns the string concatenation of all provided parameters using the `glue` between the values, empty values (`null`, `false`, `""`) are skipped

---

gt(*mixed* val1, *mixed* val2): *boolean*

returns true if val1 is greater than val2

---

gte(*mixed* val1, *mixed* val2): *boolean*

returns true if val1 is greater than or equal to val2

---

html_entity_decode(*string* input): *string*

decodes HTML entities in the `input` string

---

if(*boolean* condition, *mixed* true_value, *mixed* false_value): *mixed*

returns `true_value` if `condition` is true and `false_value` otherwise

---

impersonator_user(*string* tag, *bool*|*string* format = false): *mixed*

returns the `tag` value from the user that impersonates the current user. The impersonator is resolved in this order:

1. Impersonated admin
2. Impersonated manager
3. Impersonated cooperator

When nobody impersonates the current user, the current user is used.

---

implode(*array* value, *string* glue = *", "*): *string*

concatenates all values from array `value` using the `glue` between the values

---

import_data(*string* import_tag, *string* id, *string* key = *null*): *mixed*

return the `key` value of the import data row `id` stored under `import_tag`, or the whole row data when `key` is empty

---

in_array(*array* input, *mixed* value, *bool* strict = false): *bool*

returns true if the `value` is in the `input` array, if `strict` is true, the comparison is strict

---

index(*array* value, *mixed* index = 0): *mixed*

returns the item from array `value` stored under `index`, the `index` can be a plain index from the array or a dot notation for nested items

---

instance(*string* tag, *bool*|*string* format = *false*): *mixed*

returns `tag` value from the active `instance`

---

intermediate_report_count(*bool* exclude_rejected = false): *int*

return the number of submitted intermediate reports for the active application. When `exclude_rejected` is true, rejected reports are not counted.

---

item(*mixed* key, *mixed* value): *array*

create an array item from `key` and `value`, if `key` is `null`, add under key `0`. This function is useful when used together with the array function, to create key-value pairs.

---

json_decode(*string* input): *mixed*

decode the JSON string `input`, objects are returned as arrays; returns null for invalid JSON

---

lowercase(*string* input): *string*

return lowercase version of the `input` string. If `input` is not a scalar, an empty string is returned.

---

lt(*mixed* val1, *mixed* val2): *boolean*

returns true if val1 is less than val2

---

lte(*mixed* val1, *mixed* val2): *boolean*

returns true if val1 is less than or equal to val2

---

map(*mixed* index, *array* value = *[]*): *mixed*

returns the item from array `value` stored under `index`, or `false` when there is none

---

match(*mixed* val1, *mixed* val2): *boolean*

returns true if the values val1 and val2 are equal after normalization (lowercase and removed spaces)

---

max(*numeric* val1, *numeric* val2, ...): *numeric*

returns the maximum of all provided values

---

md5(*string* input): *string*

return MD5 hash of the input string

---

min(*numeric* val1, *numeric* val2, ...): *numeric*

returns the minimum of all provided values

---

mul(*numeric* val1, *numeric* val2, ...): *numeric*

returns the multiplication of all provided values

---

neq(*mixed* val1, *mixed* val2): *boolean*

returns true if the values val1 and val2 are not equal

---

new(*string* tag, *boolean* format = *true*): *mixed*

returns the `tag` value on the NEW side of the diff being rendered (the state after the change), formatted unless `format` is `0`

available while a DIFF is generated - a change-form or application-version diff view/PDF, or a DIFF output template

---

nl2br(*string* input): *string*

inserts an HTML `<br />` tag before each newline in the `input` string

---

nospaces(*mixed* value): *string*

removes all spaces from `value`

---

not(*mixed* value): *boolean*

returns the logical negation of `value`

---

now(): *timestamp*

returns the current time, can be then used in other functions like date, timeAdd

---

number(*mixed* value): *numeric*

converts `value` to number, even if it contains non numeric character, eg `125%`

---

number_format(*float* number, *int* decimals = 0, *string* decimal_separator = ".", *string* thousands_separator = ","): *string*

formats a number, see https://www.php.net/manual/en/function.number-format

---

old(*string* tag, *boolean* format = *true*): *mixed*

returns the `tag` value on the OLD side of the diff being rendered (the state before the change), formatted unless `format` is `0`

available while a DIFF is generated - a change-form or application-version diff view/PDF, or a DIFF output template

---

or(*boolean* val1, *boolean* val2, ...): *boolean*

returns the logical OR from all input arguments, a `null` argument ends the list

---

payment_iban_verified(*string* payer_iban, *array* statement_files, *string* recipient_iban): *bool*

Returns a previously cached successful payment verification. It deliberately does not start OCR or AI processing, so it is safe to use as a form item's fill condition.

---

pdf_page_count(*string* html, *string* css = null): *int*

returns the number of pages in the PDF generated from `html`, if `css` is provided, it is used as the CSS for the PDF generation otherwise the default print CSS is used

---

percent(*numeric* value, *numeric* percent, *numeric* precision = *2*): *numeric*

counts the provided `percent` from `value` with `precision` decimal places

---

program(*string*|*int* program_identificator, *string* tag, *bool*|*string* format = *false*): *mixed*

returns the `tag` value from program with tag or ID `program_identificator`

---

program_config(*string* key, *mixed* default = null, *int* program_id = null): *mixed*

get custom setting value of `key` for program. If program_id is not specified, program for the current application is taken

---

regexp(*string* regexp, *mixed* val): *boolean*

returns true if the value val matches the regexp

---

remove_newlines(*string* input): *string*

removes all newline characters from the `input` string

---

render_form_item(*string* tag, *array* data, *enum(view,print,pdf)* context = view, *int* formInstanceId = null): *string*

renders the form item with `tag` using the provided `data` and `context`. If `formInstanceId` is provided, it is used to find the form instance, otherwise the main application is used.

---

replace(*string* haystack, *string* search, *string* replace): *string*

replace all occurrences of `search` with `replace` in `haystack`

---

roller_value(*string* roller, *string* valueCode, *string* lang = null): *string*

return the value for the specified `roller` and `valueCode`, if `lang` is provided, the value is returned for that language instead of the current language

---

roller_values(*string* roller, *string* lang = null): *array*

return all active values for the specified `roller` as an array of `['code' => ..., 'value' => ...]` pairs, if `lang` is provided, the values are returned for that language instead of the current language

---

round(*numeric* value, *numeric* precision = *0*): *numeric*

rounds `value` to `precision` decimal points

---

rows_added(*string* tag): *array*

returns the rows added to the `tag` table (or repeatable sub-section) between the two sides of the diff being rendered (each row as an associative array of its columns)

available while a DIFF is generated - a change-form or application-version diff view/PDF, or a DIFF output template

---

rows_changed(*string* tag, *boolean* details = *false*): *array*

returns the changed rows of the `tag` table (or repeatable sub-section) between the two sides of the diff being rendered, each as an `old` / `new` pair of the row's columns; with `details` each side is instead `key` (the row's key on that side, for tag paths like `tag[key].column`), `row` (the columns) and `changed` (column tag => whether that cell changed; a changed container column such as a table popup section maps its own children the same way)

available while a DIFF is generated - a change-form or application-version diff view/PDF, or a DIFF output template

---

rows_removed(*string* tag): *array*

returns the rows removed from the `tag` table (or repeatable sub-section) between the two sides of the diff being rendered (each row as an associative array of its columns)

available while a DIFF is generated - a change-form or application-version diff view/PDF, or a DIFF output template

---

seq(*mixed* val1, ...): *mixed*

evaluates all input parameters one by one and returns the last returned value, useful with the function assign

---

set_instance_custom_data(*string* key, *mixed* value): *void*

sets the instance data stored under `key` to `value`. `key` is a dot notation key for nested data.

---

spell(*numeric* value, *string* currency = *"EUR"*, *string* language = *"sk"*, *string* fractionFormat = null): *string*

return a word representation of `value`. If `fractionFormat` is provided, the fraction is not appended but formatted

---

strip_tags(*string* input): *string*

removes all HTML tags from the `input` string

---

strlen(*string* source): *int*

returns the length of the provided string

---

strtotime(*string* value): *timestamp*

converts string representation of time to timestamp value, which can then be used in date, timeAdd

---

sub(*numeric* val1, *numeric* val2, ...): *numeric*

returns the first value minus all the following values, non-numeric values are ignored

---

substr(*string* haystack, *int* index = *0*, *int* length = *null*): *string*

returns the substring of length `length` from `haystack` starting at index `index`, an empty `length` returns the rest of the string

---

sum(*numeric* val1, *numeric* val2, ...): *numeric*

returns the `SUM` of provided values, an array value adds the sum of its numeric items

---

tag(*string* tag, *bool*|*string* format = *false*): *mixed*

returns the evaluated tag, for *format* use 0 for false and 1 for true

A value holding several things - the checked options of a MultiValue item, one column read across the rows of a table or a repeated section - is written out as one text (`"Áno, Nie"`) when it is formatted, which is what a template wants and what anything reading the value as data does not. Such a reader names the form it wants instead of passing `1`:

- `"list"` keeps the values apart: `["Áno", "Nie"]`
- `"map"` keeps them against what they are stored as: `{"a": "Áno", "b": "Nie"}`

Everything else reads the same in all three: a plain Select is its label, a date is a date. Every tag function taking a *format* accepts these.

---

timeAdd(*mixed* time, *int* days = *0*, *int* months = *0*, *int* years = *0*): *timestamp*

add the specified `days`, `months` and `years` to the provided `time`

---

time_year_array(*date* from, *date* to): *array*

returns the list of years between `from` and `to` as array

---

translate(*string* text, *string* targetLanguage = *active_language()*, *string* sourceLanguage = *null*): *string*

translates `text` to `targetLanguage` using AI. If `targetLanguage` is not provided, the active user language is used. The `sourceLanguage` is auto-detected when omitted. On failure the original `text` is returned unchanged.

---

uppercase(*string* input): *string*

return uppercase version of the `input` string. If `input` is not a scalar, an empty string is returned.

---

user(*int* id, *string* tag, *bool*|*string* format = *false*): *mixed*

returns the `tag` value from user with `id`

---

uuid(): *string*

generates a new UUID

---

val(*mixed* value): *mixed*

identity function, returns the provided value or `0` if the value is not provided

---

validation_report_url(): *string*

URL of the manager's Validačný report for the application this form hangs off, or an empty string when the viewer may not see it.

The report is gated by MANAGE_ADMINISTRATIVE_CHECK, so an applicant filling the same form gets nothing back and the link simply does not render — the check is repeated here rather than relying on the item being hidden.

---

validation_state(*string* method = *''*): *string*

state of the newest on-demand validation run of the form instance being filled: `pending`, `running`, `done`, `failed`, or an empty string when it was never started. With `method` given (e.g. `validate_expenses`) only runs of that method are considered.

Meant for a VALIDAČNÝ PRVOK: return an empty string while it runs to leave the item pending without a red box, and a message once it fails.